ERP Ze Tech

Documento de Requisitos Técnicos (TRD)

Versão	1.0
Data	16/10/2026
Situação	Baseline técnico após a Fase 0 e a base da NFC-e
Repositório	felipeaugusantos/erpzetech
Documento par	PRD — Requisitos de Produto

Sumário

1. Escopo e princípios
2. Arquitetura
3. Stack tecnológica
4. Estrutura do código
5. Multiempresa, papéis e filial (segurança no banco)
6. Modelo de dados e padrões do banco
7. Padrões de integridade (idempotência, travas, transações)
8. Trabalho sem internet
9. Módulo fiscal e NFC-e
10. Integrações e webhooks
11. Configuração, segredos e ambientes
12. Qualidade: testes e CI
13. Implantação e operação
14. Desempenho e escala
15. Dívida técnica e riscos
16. Decisões técnicas registradas
17. Roteiro técnico das próximas fases

> **Fontes.** Este documento descreve o que existe no repositório em 16/10/2026 (código, migrations, CI e documentos da Fase 0). Números são contagens do código naquela data. Itens que ainda não existem aparecem como *planejado*. Veja também `docs/diagnostico-fase-0.md`, `docs/captcha-turnstile.md` e `roadmap.md`.

1. Escopo e princípios

Este documento define como o ERP Ze Tech é construído: arquitetura, segurança, dados, integrações, qualidade e operação. Ele atende aos requisitos funcionais do PRD.

Princípios de engenharia

1. **O banco é a fonte da verdade das regras.** Papel, empresa, filial, saldo, numeração e andamento da nota são impostos no Postgres (RLS, funções, gatilhos). A interface só reflete.
2. **Operação inteira em uma chamada.** Fluxos de várias etapas viram uma função do banco (uma transação), nunca uma sequência de gravações do navegador.
3. **Idempotência onde há dinheiro.** Reenviar a mesma operação nunca duplica venda, estoque ou caixa.
4. **Falha segura.** Operação recusada fica guardada com o erro e nunca é descartada sozinha; verificação externa que não responde recusa o acesso (CAPTCHA).
5. **Segredos só no servidor.** Nada sensível volta ao navegador (ex.: token do CSC); a auditoria mascara segredos.
6. **Tudo que é regra é testado no CI**, e o CI reprova tabela ou função nova sem proteção.

2. Arquitetura

Navegador / PWA — React 19, TanStack Router (rotas por arquivo) e TanStack Query (cache persistido no aparelho; mutações com `networkMode: "always"`). Service worker guarda frente de caixa, salão, delivery e conferência. Filas offline no `localStorage`.

▼ chamadas autenticadas (RPC e consultas com a sessão do usuário) | server functions

Servidor da aplicação — TanStack Start (SSR + `createServerFn`) construído com Vite/Nitro (alvo Cloudflare por padrão do Lovable). Executa o que exige segredo ou terceiro: emissão fiscal (ACBr), verificação do CAPTCHA, consulta de GTIN, sugestão por IA, criação de usuários, webhooks de entrada e rotinas agendadas.

▼ cliente com a sessão do usuário (RLS) | cliente de serviço (somente servidor)

Supabase — Auth (JWT), Postgres com RLS, funções `SECURITY DEFINER`, gatilhos, índices, armazenamento de arquivos. Aqui moram as regras de negócio.

▼

Serviços externos — ACBr API (NF-e/NFC-e/NFS-e), Cloudflare Turnstile (CAPTCHA), Cosmos (GTIN), gateway de IA do Lovable, provedores de Pix/boleto/TEF (planejado) via webhooks assinados.

Dois caminhos de acesso aos dados

Caminho	Credencial	Quando usar	Garantia
Navegador → Supabase	Chave pública + JWT do usuário	Leituras e operações do dia a dia, via consulta ou função (RPC)	RLS e conferência de papel dentro da função; o usuário não ultrapassa o que o banco permite.
Servidor → Supabase (usuário)	JWT repassado pelo middleware de autenticação	Server functions que agem em nome do usuário	Mesmas regras do usuário; o servidor confere papel e empresa antes de chamar terceiros.
Servidor → Supabase (serviço)	Chave de serviço (nunca vai ao navegador)	Gravar retorno da Sefaz, registrar webhook, enviar contato, ações administrativas	Ignora RLS; por isso cada uso confere papel e empresa antes e é restrito a poucas funções.

3. Stack tecnológica

Camada	Tecnologia	Observação
Linguagem	TypeScript (estrito) e SQL (PL/pgSQL)	Typecheck no CI; testes unitários executam TypeScript direto no Node.
Interface	React 19, Tailwind CSS 4, Radix UI (componentes), lucide-react, sonner, recharts, react-hook-form + zod	Componentes em `src/components/ui`; layout em `AppShell`.
Rotas e dados no cliente	TanStack Router 1.170, TanStack Query 5 (com persistência), TanStack Start 1.168	Rotas por arquivo em `src/routes`; `routeTree.gen.ts` gerado.
Build	Vite via `@lovable.dev/vite-tanstack-config`, Nitro	A configuração do Lovable já inclui os plugins; não duplicar.
Backend de dados	Supabase (Postgres, Auth, Storage)	Regras no banco; tipos gerados em `src/integrations/supabase/types.ts`.
IA	SDK `ai` + gateway do Lovable	Sugestão de produtos por texto livre na frente de caixa.
PDF	jsPDF	Relatórios e arquivos de nota; cupom e romaneio por impressão do navegador.
Fiscal	ACBr API (OAuth 2.0 client_credentials)	Cliente em `src/lib/acbr.server.ts`.
CAPTCHA	Cloudflare Turnstile	Verificação no servidor em `src/lib/captcha.ts`.
Qualidade	ESLint 9, Prettier, node:test, GitHub Actions	4 jobs de CI.
Hospedagem / entrega	Lovable (editor, publicação e migrations) + GitHub	AGENTS.md: não reescrever o histórico publicado.

4. Estrutura do código

Caminho	Conteúdo
`src/routes/`	Páginas por arquivo: públicas (`index`, `auth`, `contato`, `boas-vindas`, login Ze Tech), API pública de webhooks (`api.public.webhooks.$provedor.ts`) e ~104 telas autenticadas em `_authenticated/`.
`src/components/`	`app/` (AppShell, PageHeader, cupom, Turnstile, EmitirNfce...) e `ui/` (biblioteca de componentes).
`src/lib/`	Regras puras testáveis (ex.: `pendencias-fiscais`, `nfce-config`, `nfce-consumidor`, `captcha`, `webhook`, `auditoria`, `nfe-escopo`) e funções de servidor (`*.functions.ts`, `*.server.ts`).
`src/integrations/supabase/`	Clientes (navegador e servidor), middleware de autenticação, tipos gerados.
`src/hooks/`	Sessão e dados do usuário, sincronização offline.
`supabase/migrations/`	113 migrations SQL. O Lovable também grava em `drizzle/migrations/` (31); o CI aplica as duas na ordem certa.
`supabase/ci/`	`run.sh` (aplica tudo num Postgres vazio), `shim.sql` (simula roles e schema auth/storage) e `smoke.sql` (32 blocos de testes de segurança e regra).
`tests/`	31 arquivos de teste unitário (193 casos).
`public/`	`sw.js` (service worker), manifest e ícones do PWA.
`docs/`	Diagnóstico da Fase 0, verificação de papéis e isolamento (SQL), CAPTCHA, PRD e TRD.

5. Multiempresa, papéis e filial

5.1 Isolamento entre empresas

Toda tabela operacional tem `tenant_id` (padrão `current_tenant_id()`) e RLS ligada. As políticas comparam o `tenant_id` da linha com o da sessão. `has_role()` considera o tenant do perfil, e papéis sem tenant herdam o do perfil. Funções `SECURITY DEFINER` conferem o tenant dentro delas e fixam `search_path`. O papel `anon` não executa funções sensíveis. Tabelas `saas_*` da Ze Tech não têm tenant e são protegidas por `eh_saas_operador()`.

5.2 Papéis (11 + operador Ze Tech)

`administrador, gestor, vendedor, caixa, estoquista, comprador, financeiro, logistica, motorista, garcom, cozinha`. A função `exige_papel(text[])` abre cada função crítica (aprovar, cancelar, emitir nota, contas a receber, caixa, estoque, entrega). Escritas diretas de cliente foram fechadas nas tabelas onde existe função (pedidos, caixa, estoque, nota, comissão). Padrão adotado: `*_interno` para o miolo e um invólucro que confere o papel.

Operação	Papéis
Criar pedido e orçamento	administrador, gestor, vendedor, caixa
Aprovar / cancelar pedido	administrador, gestor
Separar e conferir; ajustar estoque e inventário	administrador, gestor, estoquista
Planejar entrega e rota	administrador, gestor, logística
Concluir entrega	administrador, gestor, logística; motorista só a própria
Receber pagamento e gerar contas a receber	administrador, gestor, financeiro, caixa
Emitir e transmitir nota (NF-e, NFC-e)	administrador, gestor, financeiro, vendedor, caixa
Cancelar nota; preço e custo de produto	administrador, gestor
Abrir e fechar caixa	administrador, gestor, caixa
Dados fiscais da empresa e filial	administrador
Configurar NFC-e (CSC, série)	administrador, gestor

5.3 Acesso por filial

Política **restritiva** `filial_restrita` em pedidos, caixas, depósitos, contas, notas, orçamentos, compras e devoluções, e nas tabelas filhas por herança. `filial_irrestrita()` libera administrador, gestor e perfil sem filial; `pode_filial(uuid)` decide por linha. O gatilho `filial_protege` impede mover registro para filial de outro usuário.

5.4 Isolamento do salão

Garçom e cozinha só veem sessão, cardápio, mesas e comandas. A política restritiva `salao_restrito` (`NOT eh_salao_restrito()`) está em toda tabela com RLS fora da lista liberada, e `exige_nao_salao()` abre toda função chamável. O CI falha se surgir tabela ou função nova sem a proteção.

5.5 Auditoria

Gatilhos gravam em `auditoria` (autor, entidade, operação, antes e depois): `auditar_campos` (produtos, estoques, pedidos, orçamentos, cardápio) e `auditar_mudanca` (contas, comissões, notas, empresa, filiais, configuração fiscal, papéis, configuração da NFC-e). Argumentos com `!` mascaram o valor (tokens). A tabela não aceita escrita direta; índices por empresa e data sustentam a tela `/auditoria`.

6. Modelo de dados e padrões do banco

94 tabelas e 143 funções expostas nos tipos gerados (contagem em 16/10/2026), 216 políticas e gatilhos criados nas migrations. Grupos principais:

Grupo	Tabelas (exemplos)
Identidade e empresa	tenants, profiles, user_roles, role_permissoes, empresas, filiais
Cadastros	clientes, obras, produtos (+ categorias, produto_conversoes, produto_variacoes), fornecedores, profissionais, tecnicos, motoristas, veiculos
Estoque	depositos, estoques, estoque_movimentacoes, estoque_lotes, estoque_lote_movimentos, inventarios
Comercial e logística	orcamentos, orcamento_itens, pedidos, pedido_itens, pedido_historico, pedido_ocorrencias, entregas, entrega_itens, entrega_tentativas, entrega_reagendamentos
Balcão e caixa	caixas, caixa_movimentos, tef_config, tef_transacoes, gestor_autorizacoes, credito_autorizacoes
Compras	compras, compra_itens, compra_recebimentos, compra_cotacoes, compra_cotacao_itens, nfe_entrada_produtos
Financeiro	contas_receber, contas_pagar, financeiro_baixas, contas_bancarias, banco_movimentos, comissoes, vendedor_comissao_regras
Fiscal	fiscal_config, nfce_config, nfe, nfe_itens, nfe_pedidos, webhook_eventos
Restaurante	cardapio_categorias/itens/grupos/opcoes/ficha, mesas, comandas, comanda_itens, comanda_pagamentos, restaurante_config, restaurante_zonas_entrega
Assistência e locação	os_ordens, os_itens, locacoes, locacao_itens, locacao_equipamentos
Gestão SaaS	saas_clientes, saas_lojas, saas_planos, saas_faturas, saas_contas_pagar, saas_operadores, assinaturas
Transversal	auditoria, contatos (formulário público)

Padrões

- **Funções de regra** em PL/pgSQL, `SECURITY DEFINER`, `SET search_path TO 'public'`, com `REVOKE ALL … FROM PUBLIC, anon` e `GRANT EXECUTE … TO authenticated` só onde necessário.
- **Gatilhos de proteção** (`nfe_protege_estado`, `nfce_config_protege`, `filial_protege`): impedem que um usuário logado altere por escrita direta o que só o servidor ou uma função pode alterar (situação, chave, protocolo, retorno, numeração).
- **Totais e saldos no banco**: total do pedido e da compra por gatilho/função; saldo por item da entrega com trava; estoque físico, reservado e disponível calculados no banco.
- **Migrations** versionadas por data; mudanças em funções existentes feitas por `pg_get_functiondef` + substituição, com erro se o trecho não for encontrado (evita alteração silenciosa).
- **Tipos**: gerados pelo Lovable; tipos de funções novas são acrescentados à mão e podem ser sobrescritos na próxima sincronização.

7. Padrões de integridade

Problema	Solução	Onde
Venda reenviada duplica estoque e caixa	Chave de idempotência por venda (`pedidos.chave_idempotencia`); `frente_venda` devolve a venda existente; `frente_venda_existente` deixa a tela avisar	Frente de caixa
Duas chamadas ao mesmo tempo	Trava de aconselhamento por transação (`pg_advisory_xact_lock`) por chave da nota, por pedido (NFC-e), por empresa na numeração	Importação de XML, `nfce_criar`
Número de nota repetido	Contador só alterável por função (gatilho de proteção) + índice único parcial por empresa, série e número da NFC-e	`nfe`, `nfce_config`
Gravação em etapas deixa dados pela metade	Funções únicas: `filial_extra_criar`, `orcamento_salvar`, `empresa_salvar`, lotes na compra; criação de usuário desfaz a conta se uma etapa falhar	R10 da Fase 0
Nota fiscal reescrita direto na tabela	Gatilho `nfe_protege_estado`; o servidor grava o retorno da Sefaz com chave de serviço depois de conferir papel e empresa	`nfe`
Nota de XML duplicada	Conferência por chave ou por número + série + fornecedor, com trava e mensagem "em processo de importação"	Entrada de XML
Webhook repetido	Registro único por provedor + id do evento; reprocessa só falha ou evento preso; conclusão vale só para a tentativa vigente	`webhook_eventos`
Peça da OS sem baixa	Gatilhos reconciliam estoque ao aprovar e devolvem ao cancelar	Assistência

8. Trabalho sem internet

Fluxo	Como guarda	Como envia	Se o banco recusar
Frente de caixa	Fila no navegador (por usuário), cada venda com chave única	`frente_venda` ao voltar a conexão; antes consulta `frente_venda_existente`	Fica na lista de vendas com o erro; tentar de novo ou descartar com confirmação
Salão e delivery	`zt-comandas-offline`	`restaurante_sincronizar_offline` (idempotente por `offline_id`)	Comanda marcada como recusada; nunca descartada sozinha
Conferência e lotes	Retrato de estoque e caixa	Somente leitura	—
Páginas	Service worker `zt-offline-v2` guarda as 4 telas e os arquivos do sistema (só GET)	—	—
Consultas	Cache do TanStack Query persistido; `onlineManager` começa por `navigator.onLine`	Mutações com `networkMode: "always"`	—

> **Limite conhecido.** A separação entre erro de rede e recusa do banco usa o texto da mensagem (`/fetch|network|failed/i`); uma mensagem do banco com essas palavras seria tratada como rede. Pedidos, compras, financeiro, notas, entregas e a tela do motorista não funcionam sem internet (fila offline da logística está no roadmap).

9. Módulo fiscal e NFC-e

9.1 NF-e (modelo 55) — implementado

1. `gerar_nfe` / `gerar_nfe_agrupada` (banco) montam a nota a partir do pedido: emitente (empresa/filial), destinatário, itens e pendências (NCM, CFOP, CST/CSOSN, certificado, emissor). `nfe_calcular_impostos` calcula ICMS (interno/interestadual pela UF), ST, PIS, COFINS e ISS pela configuração da filial.
2. `transmitirNfe` (servidor) confere papel e empresa (`exigirPapelFiscal`), monta o JSON e chama a ACBr; `consultarNfe`, `cancelarNfeSefaz`, `arquivosNfe` e `statusEmissor` completam o ciclo.
3. O retorno da Sefaz é gravado com chave de serviço; o usuário não altera situação, chave, protocolo ou retorno diretamente.

Ambiente de homologação/produção: o token da conta tem audiência do host de produção; o ambiente é escolhido pelo campo `tpAmb` de cada nota.

9.2 NFC-e (modelo 65) — base pronta, transmissão pendente

Peça	Estado	Detalhe técnico
Configuração	**Implementado**	`nfce_config` (1 por `fiscal_config`): série, próximo número, `csc_id`, `csc_token`, contingência manual. Sem leitura ou escrita direta pelo navegador; tela usa `nfce_config_listar` / `nfce_config_salvar`. Formato: código 1–6 dígitos, token 16–40 alfanuméricos. Token mascarado na auditoria.
Criação da nota	**Implementado**	`nfce_criar(pedido, cpf)`: idempotente (trava por pedido); rascunho refeito enquanto não tem número; reserva o número só quando pronta; emitente da filial do pedido com a empresa como reserva; recusa pedido que já tem NF-e; consumidor identificado ou não.
Campos de andamento	**Implementado**	`nfe.modelo` ('nfe','nfse','nfce'), `contingencia`, `tentativas`, `proxima_tentativa_em`; protegidos pelo gatilho; índice único parcial de número.
Interface	**Implementado**	Botão em `/frente-caixa` (cupom e lista de vendas), `/nfce-config` e `/pendencias-fiscais`.
Transmissão à Sefaz	**Bloqueado**	Server function a escrever sobre `acbr.server.ts`: montar payload modelo 65 (CSC, QR Code), enviar, tratar autorizada/rejeitada/denegada, gravar retorno com chave de serviço. **Bloqueio:** documentação da API de NFC-e da ACBr inacessível pela rede do ambiente de desenvolvimento.
Contingência e reenvio	**Planejado**	Rotina agendada que reenvia notas em contingência e sem retorno (`tentativas`, `proxima_tentativa_em`), com recuo progressivo; inutilização de numeração; evento de cancelamento.
Homologação	**Planejado**	Cliente piloto em ambiente de homologação antes de abrir.

9.3 Pendências fiscais

`/pendencias-fiscais` consulta `nfe` pelo filtro montado em `filtroNotasPendentes` (rejeitada, contingência, NFC-e em rascunho e transmitida há mais de 15 min — o prazo está no filtro para o

limite de 300 linhas não esconder notas travadas) e `eventos_pendentes()` (eventos de provedores com erro, ignorados ou presos há mais de 5 min, só da empresa do usuário).

10. Integrações e webhooks

Webhooks de entrada

Endpoint `/api/public/webhooks/<provedor>`. Verifica a assinatura HMAC (SHA-256/SHA-1, hex/base64, prefixo configurável) com segredo por provedor, registra o evento uma única vez (`webhook_registrar`) e chama o `processar` do provedor, que precisa ser idempotente. Falha devolve 500 e o provedor repete; o evento é reprocessado. Para adicionar um provedor: segredo em variável de ambiente, um `ProvedorWebhook` em `webhook.server.ts` e a URL informada ao provedor. Hoje nenhum provedor está cadastrado (aguardando a escolha de Pix/boleto/TEF).

Integração	Uso	Variáveis	Estado
ACBr API	Emissão fiscal	`ACBR_CLIENT_ID`, `ACBR_CLIENT_SECRET`	**Implementado** NF-e / **Bloqueado** NFC-e
Cloudflare Turnstile	CAPTCHA do contato	`VITE_TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	**Implementado** (liga ao cadastrar as chaves)
Cosmos	Consulta de GTIN	`COSMOS_TOKEN`, `COSMOS_USER_AGENT`	**Implementado**
Gateway de IA do Lovable	Sugestão de produtos	`LOVABLE_API_KEY`	**Implementado**
Agendamentos do Lovable	Rotinas autenticadas por segredo	`LOVABLE_CRON_SECRET` (e o anterior, para rotação)	**Implementado**
Pix / boleto	Cobrança e baixa por webhook	`PIX_WEBHOOK_SECRET` (reservado)	**Bloqueado** escolha do provedor
TEF	Maquininha real	—	**Planejado** (modo simulado existe)
iFood	Pedidos do delivery	—	**Bloqueado** credencial de parceiro
Extrato bancário	OFX/API na conciliação	—	**Planejado**

11. Configuração, segredos e ambientes

Variável	Onde roda	Para quê
`SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`	Servidor e navegador (`VITE_*`)	Conexão com o Supabase usando a sessão do usuário
`SUPABASE_SERVICE_ROLE_KEY`	**Somente servidor**	Cliente de serviço (gravar retorno fiscal, webhooks, contato, administração)
Demais (tabela da seção 10)	Somente servidor, exceto a chave pública do Turnstile	Integrações

- Segredos ficam nas variáveis do projeto (Lovable) e nunca no repositório; o servidor lê `process.env` em tempo de execução.
- Sem as chaves do Turnstile o formulário de contato segue como antes; com a chave secreta definida e a Cloudflare indisponível, o envio é recusado (falha segura). As duas chaves devem ser cadastradas juntas.
- O IP do visitante é usado só para limitar envios e é guardado como resumo irreversível (`md5('contato:'||ip)`).
- Ambientes: o editor/preview do Lovable e a publicação; o CI roda sobre um Postgres vazio criado a cada execução.

12. Qualidade: testes e CI

Job do CI	O que faz	Falha quando
Migrations e segurança	Aplica todas as migrations (SQL e drizzle, na ordem) num Postgres vazio e roda `smoke.sql` (32 blocos)	Papel/tenant/filial vazando; tabela ou função sem proteção do salão; regra de negócio quebrada
Testes unitários	`node --test --experimental-strip-types` (31 arquivos, 193 casos)	Regra pura quebrada (impostos, XML, webhook, CAPTCHA, auditoria, NFC-e…)
Typecheck e build	`tsc --noEmit` e `vite build`	Erro de tipo ou de build
Lint	ESLint (0 erros; 62 avisos conhecidos) com Prettier	Erro de lint ou formatação

- **Fluxo de trabalho:** uma branch por mudança, PR com revisão automática (Codex); achado é tratado como trabalho, resposta na conversa antes do push e correção testada; merge por squash.
- **Testes do banco** executam cada cenário com o papel certo (função `ci.entrar(usuário)`), verificam recusas (`ci.deve_falhar`) e invariantes (`ci.exige`).
- **Verificação de papéis e tenant:** o script `docs/fase-0/verificacao-papeis-e-tenant.sql` repete a conferência contra o banco real.
- **Lacuna:** não há teste automatizado de interface (navegador); validação visual é manual.

13. Implantação e operação

1. O código vai ao GitHub; o Lovable sincroniza e publica. Migrations em `supabase/migrations/` são aplicadas pelo Lovable no banco do projeto.
2. **Ordem importa:** as migrations da Fase 2 dependem umas das outras (`20261016000000` base da NFC-e → `…01` tela de configuração → `…02` emitente da filial). A migration do CAPTCHA (`20261015000008`) deve entrar junto com o código, senão o formulário de contato fica fora do ar por instantes.
3. Antes de abrir a um cliente: aplicar em ambiente de teste e conferir com um usuário de cada perfil (filial, auditoria, NF-e, frente de caixa, contato).
4. **Rollback:** migrations são aditivas; reverter uma função é reaplicar a versão anterior; dados de nota não são apagados.
5. **Monitoramento:** captura de erros do front (`error-capture`, relatório de erros do Lovable); pendências fiscais e eventos de pagamento são o painel operacional. Alertas externos e métricas de desempenho: *planejado*.

14. Desempenho e escala

- Consultas de listas usam limite e ordenação por índice; auditoria tem índices por empresa e data; pendências fiscais têm índice parcial por situação.
- As políticas chamam `has_role()` e `current_tenant_id()` por linha. Está no roadmap medir o custo com dados reais e, se preciso, trocar por `(SELECT …)` (avaliado uma vez por consulta), como já feito em `salao_restrito`.
- Cache de consultas no cliente reduz chamadas e sustenta o uso offline.
- Escala horizontal: servidor sem estado (Nitro) e Supabase gerenciado; gargalos prováveis são políticas por linha e relatórios agregados. Metas de latência e volume: *a definir* com medição.

15. Dívida técnica e riscos

Item	Risco	Plano
Migrations da rodada ainda não aplicadas no banco real	Alto	Aplicar em ambiente de teste e rodar a verificação de papéis.
Tipos gerados sobrescrevem ajustes manuais	Médio	Conferir tipos a cada sincronização; avaliar gerar tipos no CI.
Lockfile: o `bun.lock` aponta para registro privado do Lovable	Médio	Lockfile reproduzível na CI.
Telas grandes (`pedidos.$id`, `compras.$id`, `frente-caixa`)	Baixo	Quebrar em componentes ao tocar.
62 avisos de lint (hooks e fast refresh)	Baixo	Limpar gradualmente.
Detecção de erro de rede por texto da mensagem	Baixo	Classificar por código de erro.
Sem teste automatizado de interface	Médio	Testes de fluxo (Playwright) para frente de caixa, salão e NFC-e.
Custo de `has_role` nas políticas	Médio	Medir e otimizar com dados reais.
Transmissão fiscal depende de terceiro e de documentação	Alto	Isolar a camada ACBr; homologar com piloto.

16. Decisões técnicas registradas

#	Decisão	Motivo
D1	Regras de negócio no Postgres (funções, gatilhos, RLS)	Uma só aplicação das regras para navegador, servidor e integrações; testável no CI.
D2	Papéis verificados dentro de cada função crítica (`exige_papel`)	Interface não é fronteira de segurança.
D3	Política restritiva para filial e para o salão	Soma-se às políticas de tenant sem reescrevê-las.
D4	Retorno fiscal gravado só pelo servidor, com gatilho de proteção	Quem emite não pode reescrever situação, chave ou protocolo.
D5	CSC e tokens sem leitura direta pelo navegador; acesso por funções; auditoria mascarada	Reduz o dano de um navegador comprometido.
D6	Idempotência por chave gerada no cliente para vendas offline	Reenvio não duplica dinheiro nem estoque.
D7	Operação recusada nunca é descartada sozinha	O dinheiro já foi recebido no balcão.
D8	Vendedor e caixa emitem nota do balcão; só administrador e gestor cancelam	Decisão do dono, sem abrir escrita direta nas tabelas de nota.
D9	Acesso por filial com exceção de administrador, gestor e perfil sem filial	Decisão do dono.
D10	NFC-e: número reservado só quando a nota está pronta; rascunho é refeito	Evita buracos na sequência fiscal.
D11	Webhooks com assinatura, registro único e processamento idempotente	Provedores repetem entregas.
D12	Migrations aditivas, com ajustes em funções por substituição verificada	Evita alteração silenciosa ao editar funções longas.

17. Roteiro técnico das próximas fases

Fase	Trabalho técnico	Dependência
2 (continuação)	Server function de transmissão da NFC-e (payload modelo 65, QR Code, CSC); rotina de reenvio e contingência; inutilização; botão no PDV de pedidos e no restaurante; webhooks de Pix e boleto sobre a estrutura existente; baixa automática no financeiro.	Documentação da ACBr; provedor de Pix/boleto
1	Importação por planilha: leitura no navegador, validação por linha no servidor, gravação transacional com limite por empresa, relatório de erros.	—
3	Adaptador de TEF por credenciadora sobre `tef_transacoes`; importação OFX e conciliação.	Credenciadora
4	Geração de SPED e exportações a partir das tabelas fiscais; regras por UF.	Notas estáveis; contador
5	API pública com chaves por empresa, webhooks de saída (fila e reentrega), conectores de WhatsApp e marketplace.	Auditoria e idempotência (já existem)
Transversal	Testes de interface, lockfile reproduzível, tipos gerados no CI, métricas e alertas, otimização de políticas.	—